



AI FINAL ASSIGNMENT — ARTIFICIAL INTELLIGENCE, 3RD YEAR COMPUTER SCIENCE

PhishGuard

A machine-learning phishing URL detector — an explained verdict from the URL string alone, running entirely in your browser.

Anayo Chibuikwe Anyafulu • Student No. 202401813

Course: Artificial Intelligence (3rd Year, Computer Science) | Lecturer: Eng. Marciano Ombe

Problem, Motivation & Objectives



The problem

Zero-day phishing outpaces blocklists. The URL is the one artefact every attack must create — so it is a detection surface that always exists, before the click happens.



The motivation

Pre-click detection that is instant, offline and privacy-preserving — embeddable anywhere a URL appears, with nothing sent over the network.

PROJECT OBJECTIVES



Reproducible ML pipeline

Train and compare at least two model families on real, current URL data.



Lexical features only

No network access at inference — instant, offline, privacy-preserving detection.



Browser deployment

Inference runs client-side, with a per-verdict explanation the user can read.



Honest evaluation + ethics

Held-out test set, cross-validation, leakage checks, and disclosed trade-offs.

Background, in One Slide



Lexical detection is competitive

Simple, interpretable URL features remain deployable and competitive with heavier approaches.

Sahingoz et al., 2019



Explainability builds trust

Rule-extraction work shows explainable verdicts matter as much as raw accuracy.

Mohammad et al., 2015



Most models never ship

The deployment gap: published detectors routinely depend on network lookups and stay in the notebook.

Basit et al., 2021



Accuracy vs. transparency

Tree ensembles win on accuracy; linear models win on transparency — a real trade-off, not a flaw.

Breiman, 2001; Cox, 1958

Model Architecture & Design

18 lexical features extracted from the raw URL string — lengths, character counts, entropy, IP-host and TLD-in-subdomain checks, shortener lookups — computed identically in Python and TypeScript.



RandomForest

200 trees • accuracy benchmark

Captures non-linear feature interactions a linear model cannot. Too heavy to export to a browser (53 MB) — stays server/CLI-side as the accuracy ceiling.

vs



LogisticRegression

2 KB export • deployable product

Means, standard deviations, weights and bias export losslessly. The browser reproduces Python's probability exactly — same formula, same digits.

$$p = \text{sigmoid}(\text{bias} + \sum w_i \cdot (x_i - \text{mean}_i) / \text{std}_i)$$

Exact arithmetic — 6-decimal Python ↔ browser parity, enforced by golden-master tests

Same 18 features, same order, in both languages — pinned by unit tests on both sides plus cross-side golden-master tests, so training-time and inference-time features can never silently drift apart.

Data & Preprocessing

107,355

real URLs trained on

48,338

PhishTank phishing URLs

59,017

legitimate (Tranco + benign deep links)

THREE FIXES THAT MADE THE DATA HONEST



Fix 1 — Path-balanced legitimate data

Tranco alone is bare homepages only. Without benign deep links, the model learns 'has a path = phishing' — dataset bias, not signal.



Fix 2 — Max 10 URLs per host

One PhishTank campaign contributed 5,700+ URLs on a single host. Capping stops the model memorising hosts instead of transferable patterns.



Fix 3 — Dedupe + stratified 80/20 split

Exact and cross-class deduplication, then a stratified split — zero duplicate URLs crossing the train/test boundary.

Functionality & Live Demo



Website checker (live demo)

Paste a URL → verdict, probability gauge, feature-occlusion explanation, full 18-feature grid — all running in the browser.



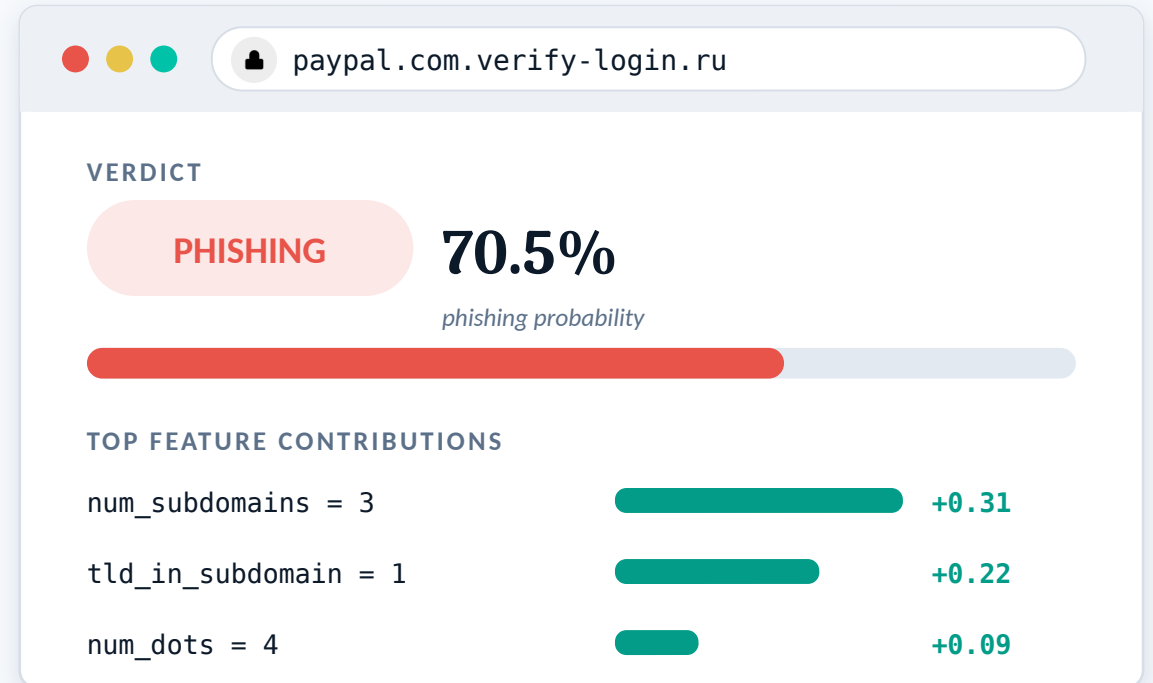
Sample URLs + history

One-click samples for both classes; a session history of recent analyses.



CLI, for scripted use

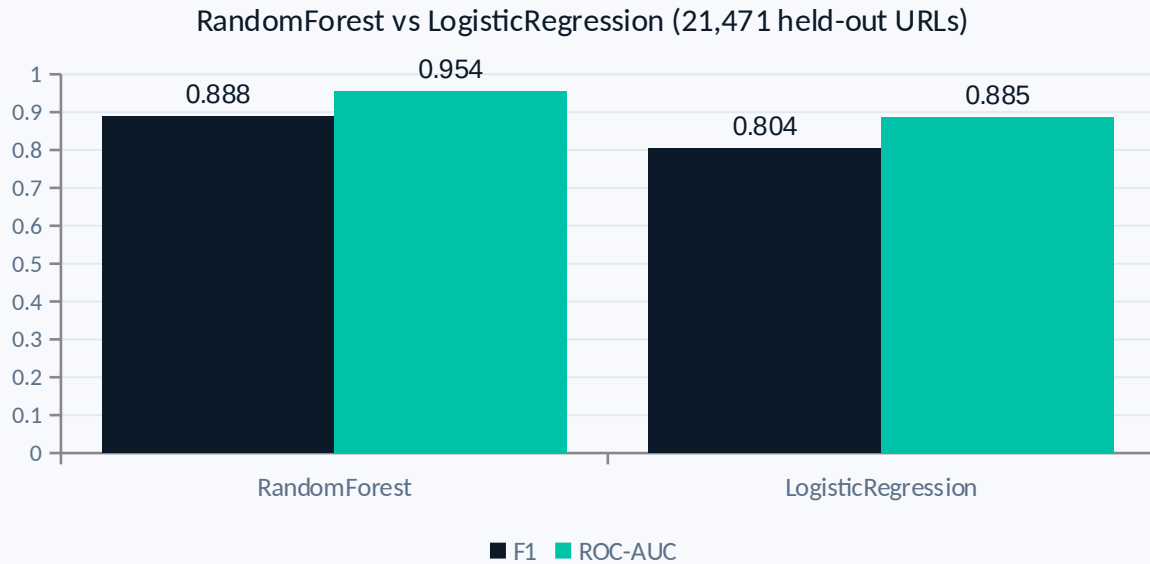
The same model is also reachable from the command line, with human and JSON output modes.



Live demo: google.com vs. paypal.com.verify-login.ru

RESULTS

Evaluation & Analysis



HONESTY CHECKS



No split fluke

CV F1 tracks test F1 closely (0.889 vs 0.888 for RF) — the result isn't a lucky draw.



Errors are balanced

Confusion matrix: FN 1,091 vs FP 1,076 — mistakes aren't collapsed into one class.



Top features are plausible

hostname_length 17.5%, path_length 17.4%, url_entropy 14.1% — inspectable, not a black box.

Key Findings & Improvements



Deployment shapes model choice

Ship LogisticRegression — 2 KB, exact, explainable. Keep RandomForest as the accuracy benchmark it can never replace in the browser.



Data hygiene beats hyperparameter tuning

Per-host capping and path-balanced data produced larger, honest gains than any tuning pass attempted.



Explanation is a feature, not a footnote

Occlusion contributions turn 'the model says 0.82' into a specific, teachable argument about why.



What's next

A bundled Public Suffix List, in-browser tree export, and active learning from user overrides.

Ethical Considerations & Mitigations



False negatives cost more than false positives

Recall is prioritised, the threshold is user-adjustable, and that choice is disclosed in-product — not buried.



Bias is named, not hidden

Multi-source data and per-host capping reduce the loudest skews; remaining limitations are stated openly.



No auto-blocking

Verdicts are advisory and overridable — a human stays in the loop for every decision.



Privacy by architecture

100% client-side inference. The URL — often carrying tokens or search terms — never leaves the browser.



CONCLUSION

An end-to-end AI system

Real data → two models → honest evaluation → deployed, explained product.

Problem framing

Feature engineering

Model selection

Evaluation discipline

Ethics

Every claim is reproducible — one command retrains, re-evaluates and re-exports everything, so every number here regenerates from source.

Thank you — questions welcome